

23CSE373 · COMPUTER VISION · GROUP 10

Fast Object Proposal Generation Using Handcrafted Gradient Features

*A from-scratch re-implementation and study of BING —
Binarized Normed Gradients for Objectness Estimation
(Cheng, Zhang, Lin, Torr · CVPR 2014)*

Abhinav Dileep AM.SC.U4CSE23202

Ayyappadas M T AM.SC.U4CSE23209

Hari Sankar A AM.SC.U4CSE23221

Parthiv M AM.SC.U4CSE23240

Sai Kishen K M AM.SC.U4CSE23248



Why object proposals?

Object detection is expensive. A sliding-window detector must classify every position $\times$ scale $\times$ aspect ratio, which is 10^5 – 10^6 windows per image.

Most windows are background. Only a handful of windows actually contain an object.

Object proposals. A cheap, class-agnostic objectness score ranks the windows, so the detector only examines the top ~ 1000 .

Base paper: BING (CVPR 2014). Handcrafted gradient features + bit operations produce proposals at 300 fps, with no deep learning.

$\sim 10^5 +$

windows a naïve sliding-window scan evaluates per image

~ 1000

proposals that a detector needs to check instead

300 fps

BING's reported speed in the paper's C++ implementation

Problem definition

Given an image I , output a ranked list of N bounding boxes such that as many objects as possible are covered ($\text{IoU} \geq 0.5$) by a small N , in milliseconds, using only handcrafted features.

1

Input

An RGB image of any size, any content

2

Output

Boxes (x_1, y_1, x_2, y_2) ranked by objectness score, plus an objectness heat-map

3

Constraints

Class-agnostic, real-time, CPU only, no deep networks

4

Success measures

Detection rate vs #proposals (DR@N), MABO (box tightness), runtime (fps)

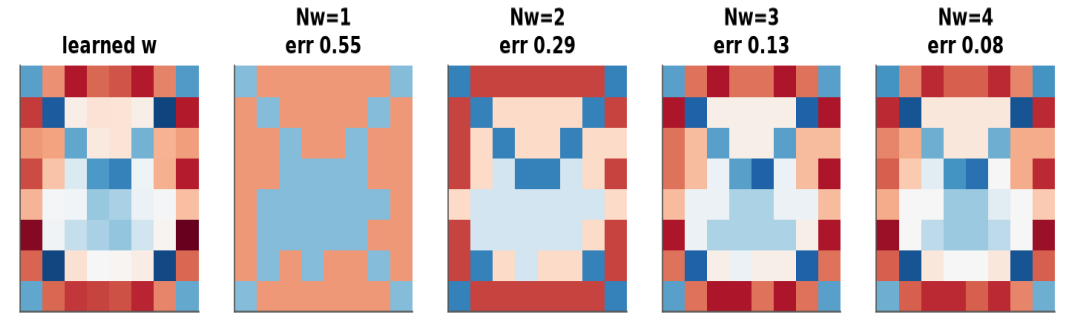
Scope: we answer “Which regions are likely to contain objects?”, not “What is the object?” (classification is a later stage).

Proposed solution: BING, rebuilt and extended

Key idea. Objects have closed boundaries. Shrink any window to 8×8 and look at gradient magnitudes: object windows show a ring of edges.

64-weight linear model. An SVM filter learns this “closed boundary” template (right); a second stage calibrates each window size.

Bits instead of floats. Filter and features are binarized, so each window costs a few AND + POPCOUNT operations.



Our novelty beyond the paper

From-scratch Python

No reuse of the C++ code;
Numba JIT kernel of the
paper's Alg. 2

New benchmark

Synthetic scenes with
clutter + 58 hand-labelled
real objects

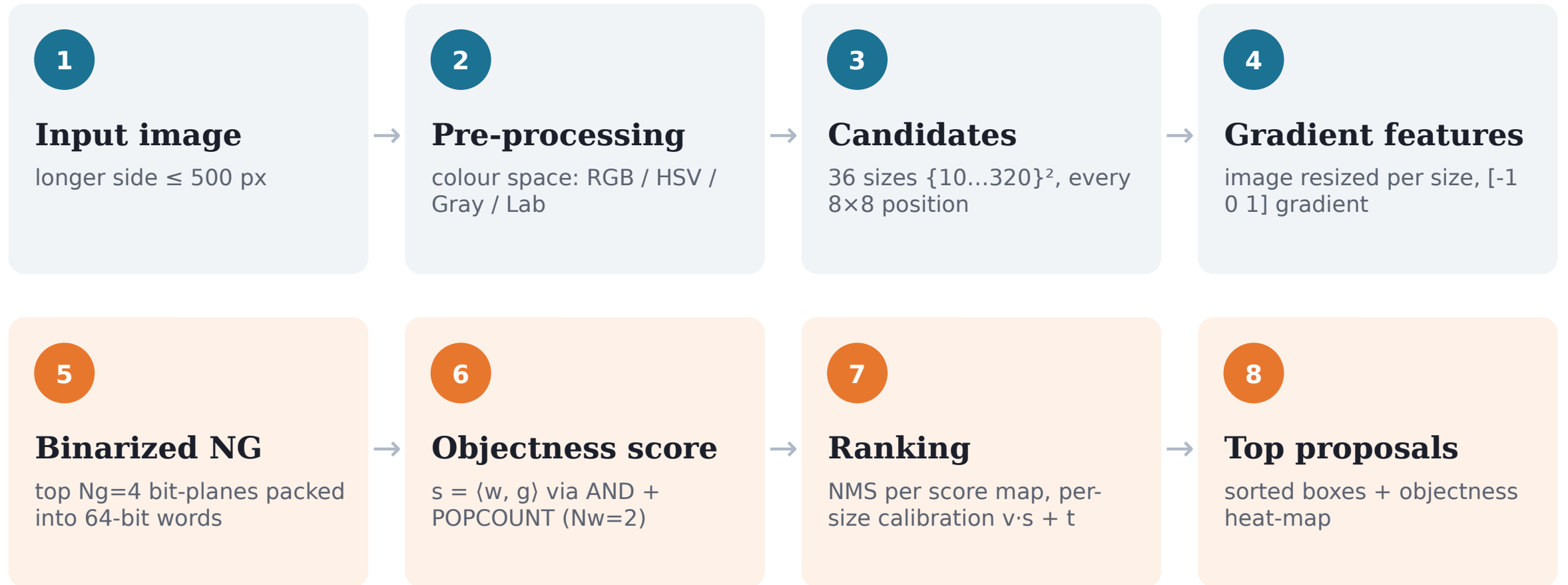
Diversified BING

Merges RGB + HSV + Gray
models → higher recall

Systematic ablations

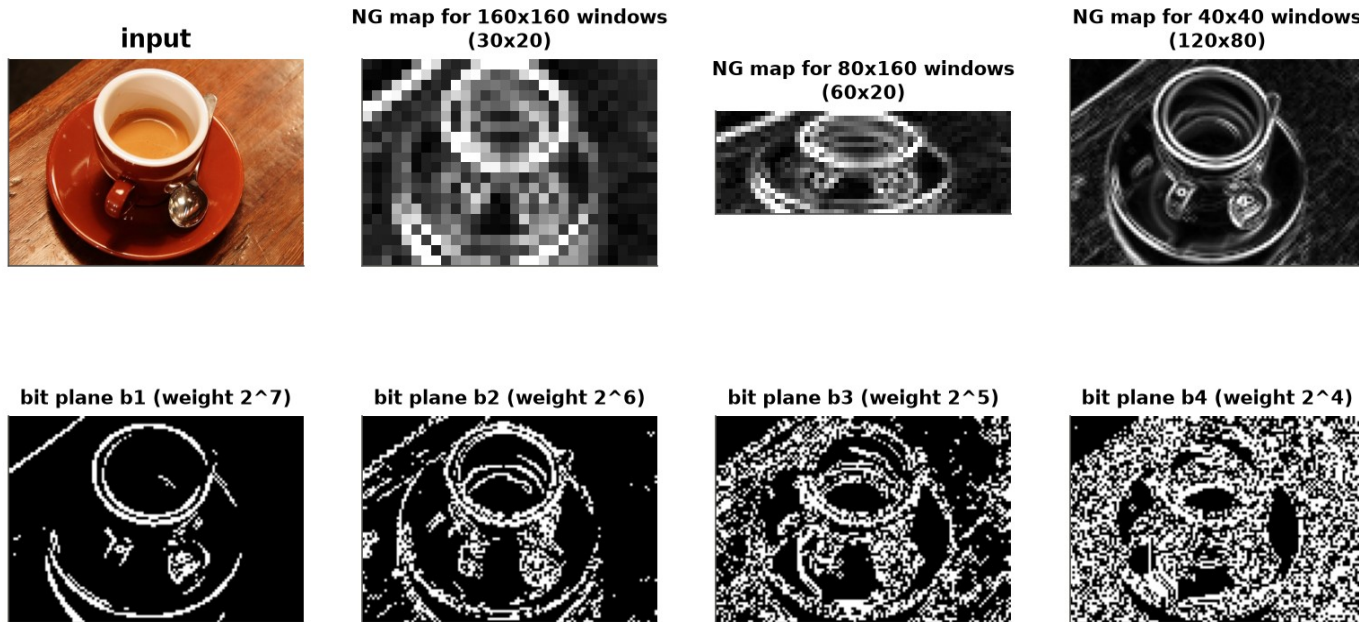
Gradient mask, colour
space, bits Nw/Ng, budget,
data size

System design: 8-stage pipeline



Training: stage I is a linear SVM on 8×8 NG features of object boxes vs random background windows; stage II is a 1-D calibration per window size.

Normed gradients → bit operations



Top: NG maps for three window sizes (each 8×8 patch = one window). Bottom: the 4 bit-planes forming the binarized feature.

- Implemented in Python: `bing/` package (features, binary, Numba fast path, model, metrics, datasets); unit tests prove bitwise score = float score.

Approximate the filter

$$w \approx \sum_j \beta_j a_j, \quad a_j \in \{-1, +1\}^{64}$$

Approximate the feature

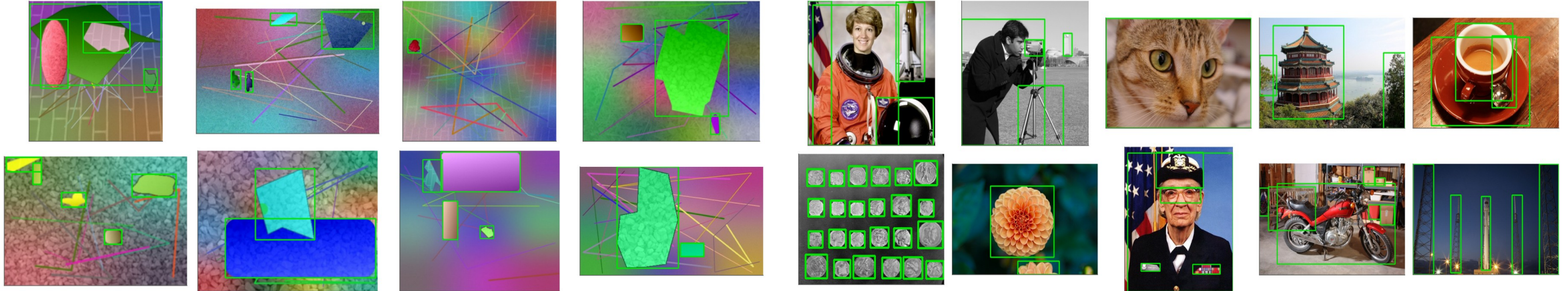
$$g \approx \sum_k 2^{(8-k)} b_k, \quad b_k \in \{0, 1\}^{64}$$

Score with bits only

$$\langle a_j, b_k \rangle = 2 \cdot \text{popcnt}(a_j^+ \text{ AND } b_k) - \text{popcnt}(b_k)$$

12 POPCOUNTs per window instead of 64 multiply-adds.

Experimental setup



Synthetic benchmark

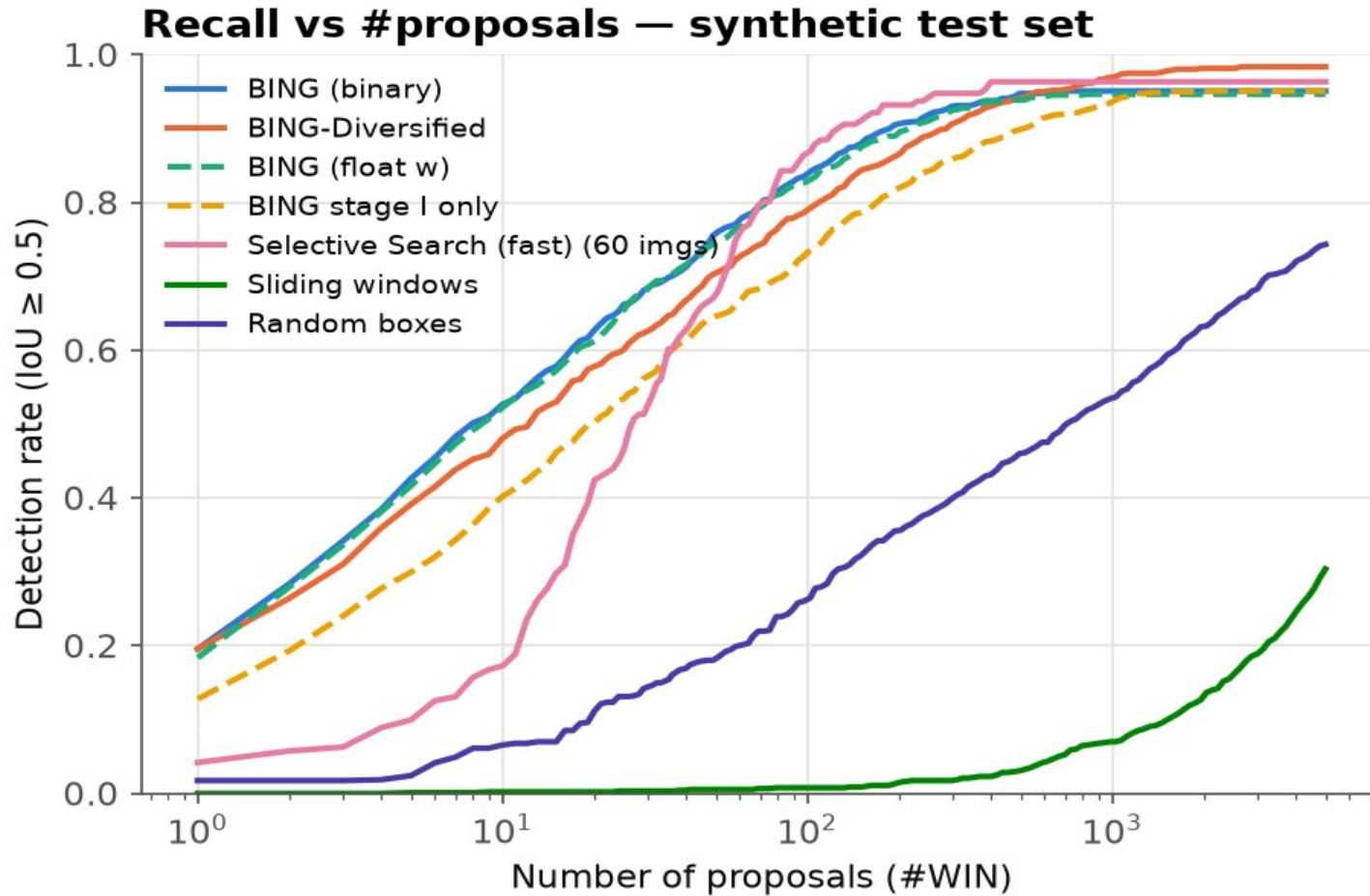
- 600 train / 300 test images, 915 test objects
- Textures, shading, noise + clutter lines (edges that are not objects)

Real photographs

- 10 natural images, 58 hand-labelled objects
- Never used for training: tests generalisation

Metrics: detection rate $DR@N$ ($IoU \geq 0.5$), MABO (mean best overlap), AUC, images/s. Baselines: Selective Search, sliding windows, random boxes. A PASCAL VOC 2007 loader is included for the paper's protocol.

Recall vs number of proposals



95.1%

objects found in the top 1000 — BING (ours)

97.0%

BING-Diversified (our RGB + HSV + Gray merge)

53.6%

random boxes, i.e. no objectness

Comparison with existing methods

Method (synthetic, 300 images)	DR@100	DR@1k	MABO@1k	img/s
BING — ours (Python + Numba)	84.0%	95.1%	0.658	160 †
BING-Diversified — ours	79.0%	97.0%	0.675	—
BING — authors' C++ code	80.2%	94.9%	0.639	104 ‡
Selective Search (fast)	88.4%	97.5%	0.895	1.4
Sliding windows	0.8%	7.0%	0.314	—
Random boxes	26.3%	53.6%	0.518	—

All methods scored with the same evaluator on the same images. † 4 threads. ‡ authors' code, 1 thread (374 img/s on 4 threads). Selective Search timed end-to-end.

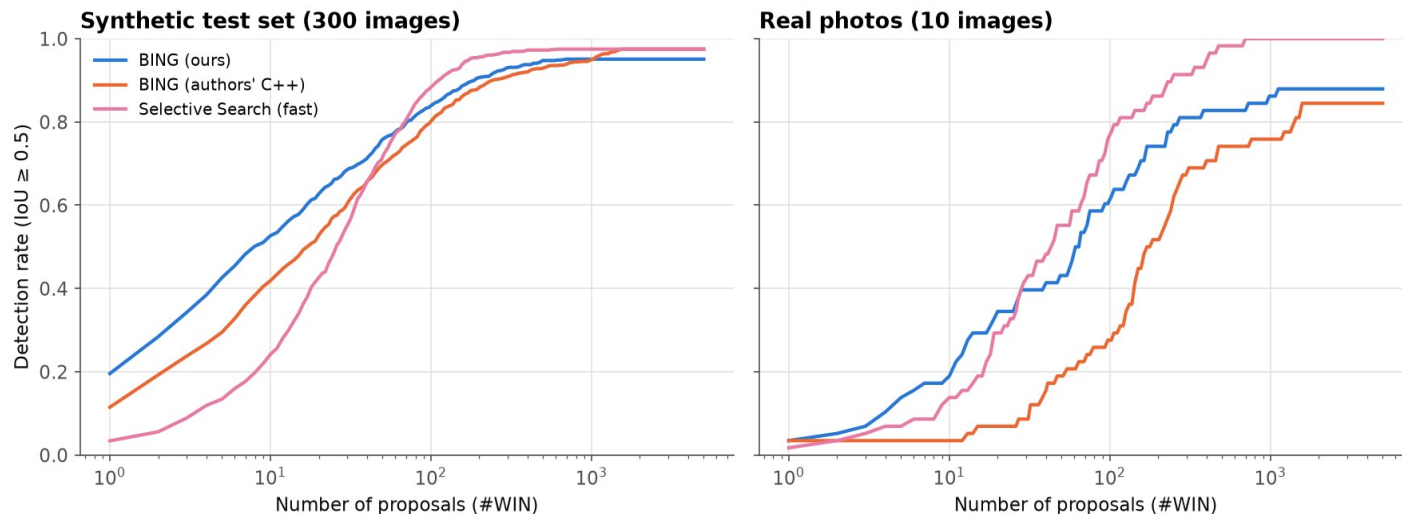
Paper vs code vs ours

- Paper (VOC 2007): DR 96.2% @1000, 300 fps
- Authors' code on our data: 94.9% @1000
- Ours on our data: 95.1% @1000

Findings

- Our rebuild matches the authors' code (+3.8 pts DR@100)
- Selective Search: tighter boxes (MABO 0.89) but ~116× slower
- Calibration adds +11 pts DR@100; binary = float accuracy

Real photos and where BING struggles

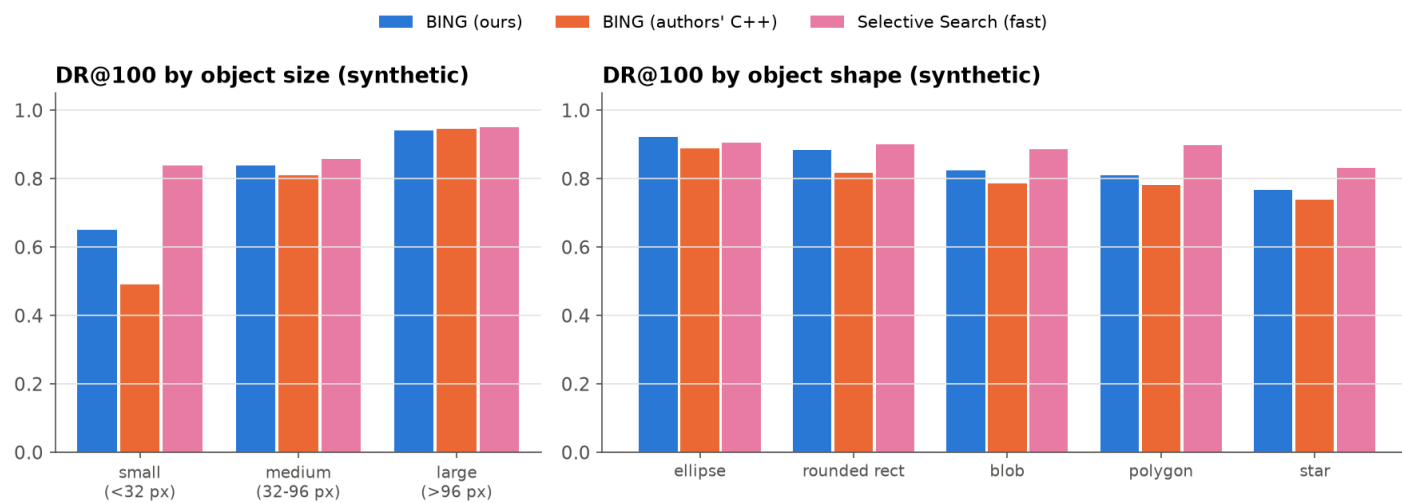


Real photos (no real training): ours 86.2%, authors' code 75.9%, Selective Search 100.0% at 1000 proposals.

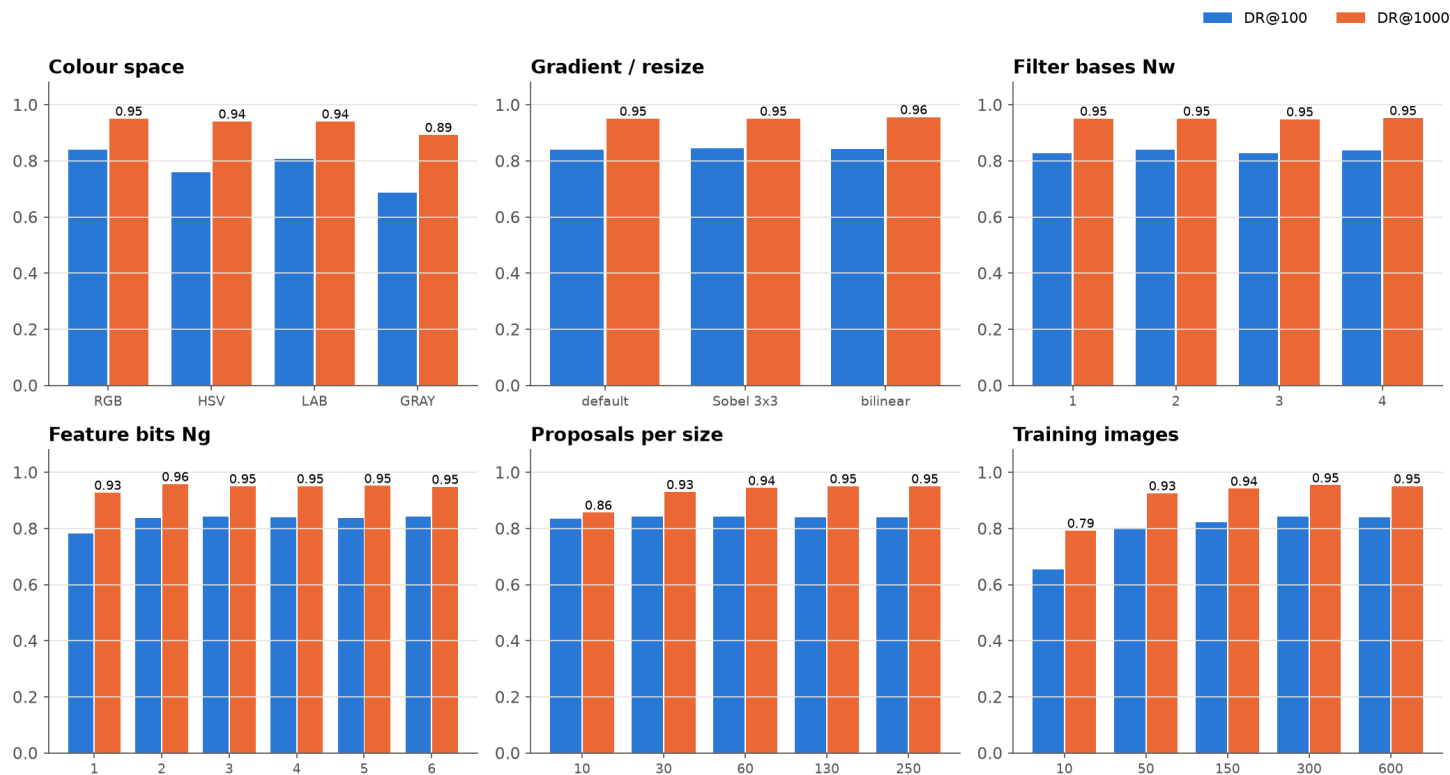
Large objects: the authors' code misses the people, cat and pagoda; ours finds them.

Small objects are hardest: DR@100 65.1% (ours) vs 49.0% (authors' code).

Star-shaped objects (thin, irregular outlines) are hardest for every method.

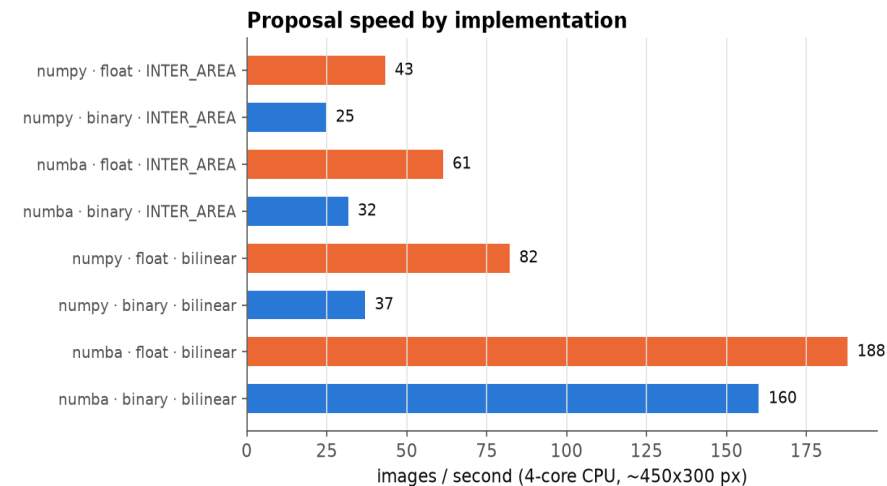


What matters? Ablations and speed



DR@1000 on the synthetic test set (bars: blue DR@100, orange DR@1000).

Limitation: power-of-two sizes and an 8-px stride give coarse boxes, so recall drops at strict IoU (MABO $\approx$ 0.66).



Gradient: $[-1 \ 0 \ 1]$ 95.1% = Sobel 95.1%

Colour: RGB 95.1% > Lab 94.1% $\approx$ HSV 94.1% > Gray 89.3%

Bits: Ng = 1 $\rightarrow$ 92.8%; Ng $\geq$ 2 $\rightarrow$ $\sim$ 95%

Data: 10 images $\rightarrow$ 79.2%; 600 $\rightarrow$ 95.1%

Speed: Numba bitwise kernel 160 img/s vs 37 in numpy

CONCLUSION

Simple gradients, a 64-weight linear model and bit operations are enough to find most objects, fast.

95.1%

objects found in top 1000
(synthetic test)

86.2%

objects found in top 1000
(real photos, no real training)

160

images / second
(bitwise, Python + Numba)

12

POPCOUNTs per window
instead of 64 multiply-adds

Future work tighter boxes (refinement / more sizes) · full PASCAL VOC evaluation · C++ SIMD kernel for 300 fps · add a classifier for a complete detector

References [1] M.-M. Cheng, Z. Zhang, W.-Y. Lin, P. H. S. Torr, "BING: Binarized Normed Gradients for Objectness Estimation at 300fps," CVPR 2014. [2] J. Uijlings et al., "Selective Search for Object Recognition," IJCV 2013. [3] B. Alexe, T. Deselaers, V. Ferrari, "Measuring the Objectness of Image Windows," TPAMI 2012. [4] J. Hosang et al., "What makes for effective detection proposals?," TPAMI 2016.

Thank you, questions?